

Hyperparameter Tuning Process

Efficiently organizing the hyperparameter tuning process is critical for converging on optimal model settings. This section outlines the hierarchy of hyperparameter importance and recommended search strategies.

1. Prioritizing Hyperparameters

Not all hyperparameters have an equal impact on model performance. They can be categorized by their relative importance during the tuning process.

- **Tier 1 (Most Critical):**
 - α (**Learning Rate**): The single most important hyperparameter to tune.
- **Tier 2 (High Importance):**
 - β (**Momentum term**): A value of **0.9** is a standard default.
 - **Mini-batch size**: Essential for ensuring the optimization algorithm runs efficiently.
 - **Number of hidden units**: Often adjusted to tune model capacity.
- **Tier 3 (Moderate Importance):**
 - **Number of layers**: Can make a significant difference but is typically tuned after the above.
 - **Learning rate decay**: Optional parameter to refine convergence.
- **Tier 4 (Low Importance - Defaults recommended):**
 - **Adam Optimization parameters**: Rarely require tuning.
 - $\beta_1 \approx 0.9$
 - $\beta_2 \approx 0.999$
 - $\epsilon \approx 10^{-8}$

2. Hyperparameter Search Strategies

Random Sampling vs. Grid Search

In earlier machine learning generations (with few hyperparameters), **Grid Search** (systematically exploring points in a grid) was common practice. However, for Deep Learning, **Random Sampling** is superior.

- **The Problem with Grid Search**: If one hyperparameter is significantly more

important than another (e.g., α vs. ϵ), grid search wastes computational resources re-evaluating the unimportant parameter while testing very few distinct values of the critical parameter.

- **The Advantage of Random Sampling:**

- Allows testing of N **distinct values** for the most important hyperparameter (where N is the number of trials).
- Provides a richer exploration of the search space, as it is difficult to know in advance which hyperparameters will be most impactful for a specific problem.
- Scales better to high-dimensional spaces (many hyperparameters).

Coarse to Fine Sampling

This strategy optimizes the search process by narrowing the search space iteratively.

1. **Coarse Search:** Perform an initial random sampling over a broad range of values.
 2. **Analysis:** Identify the region (e.g., a specific area in the hyperparameter space) that yields the best results.
 3. **Fine Search:** Zoom into that smaller region and sample more densely to find the optimal specific value.
-

Using an Appropriate Scale to Pick Hyperparameters

While random sampling is more efficient than grid search, simply sampling uniformly at random is not always optimal. The **scale** on which you explore the hyperparameter space determines how effectively you search for optimal values.

1. Linear Scale vs. Logarithmic Scale

The appropriate sampling scale depends on the nature of the hyperparameter.

- **Linear Scale (Uniform Sampling):**

- Appropriate when the range of valid values is relatively small and the sensitivity is consistent across the range.
- *Example 1: Number of Hidden Units ($n^{[l]}$).* If searching between 50 and 100, sampling uniformly is reasonable.

- *Example 2: Number of Layers (L)*. If searching between 2 and 4, explicit grid search or uniform sampling works well.
- **Logarithmic Scale:**
 - Appropriate when the hyperparameter spans several orders of magnitude, or when the model is sensitive to small changes in a specific region of the range (e.g., small values or values near 1).

2. Sampling the Learning Rate (α)

Sampling the learning rate uniformly over a wide range is inefficient because it disproportionately favors larger values.

- **The Problem:**
 - Suppose the search range is 0.0001 to 1.
 - If sampled uniformly (linear scale), **90%** of the resources are spent searching between 0.1 and 1.
 - Only **10%** of resources are allocated to the range 0.0001 to 0.1, despite this region spanning three orders of magnitude.
- **The Solution (Log Scale):**
 - Sample uniformly on the **exponents** rather than the values themselves.
 - This ensures equal resources are dedicated to the range $[0.0001, 0.001]$ as are dedicated to $[0.1, 1]$.

Implementation (Log Scale Sampling)

To sample α between 10^a and 10^b :

1. Identify the bounds in log space:
 - Low bound: $a = \log_{10}(0.0001) = -4$
 - High bound: $b = \log_{10}(1) = 0$
2. Sample a random number r uniformly between a and b :
 - $r \in [-4, 0]$
3. Calculate α :
 - $\alpha = 10^r$

Python Snippet:

```
1 | r = -4 * np.random.rand() # Generates r between -4 and 0
2 | alpha = 10**r             # Generates alpha between 10^-4 and 10^0
```

3. Sampling Beta (β) for Exponentially Weighted Averages

Hyperparameters like β (used in Momentum or Adam) require a specific transformation because their sensitivity increases dramatically as they approach 1.

- **The Range:** Suppose we want to search $\beta \in [0.9, 0.999]$.
- **The Problem:**
 - A change from $0.9 \rightarrow 0.9005$ is insignificant.
 - A change from $0.999 \rightarrow 0.9995$ is massive. It doubles the "window" of the average (from roughly the last 1,000 examples to the last 2,000).
 - The formula for the averaging window is $\frac{1}{1-\beta}$. This function explodes as $\beta \rightarrow 1$.
- **The Solution (Sampling $1 - \beta$):**
 - Instead of sampling β directly, sample the complement: $1 - \beta$.
 - If β ranges from 0.9 to 0.999, then $1 - \beta$ ranges from 0.1 to 0.001.
 - Sample $1 - \beta$ on a logarithmic scale (as detailed in section 2).

Implementation (Beta Sampling)

1. Determine range for $1 - \beta$:
 - 10^{-1} (from $1 - 0.9$) to 10^{-3} (from $1 - 0.999$).
2. Sample r uniformly between the exponents:
 - $r \in [-3, -1]$
3. Calculate β :
 - $\beta = 1 - 10^r$

Summary: This method forces the search to sample more densely in the region where β is close to 1 (or where $1 - \beta$ is close to 0), ensuring the efficient discovery of stable hyperparameters.

Hyperparameter Tuning in Practice: Pandas vs. Caviar

Strategies for organizing hyperparameter search processes generally fall into two categories, dictated primarily by the availability of computational resources.

1. General Tuning Guidelines

- **Cross-Fertilization:**

- Hyperparameter intuitions and architectures often transfer between domains.
- *Examples:* Ideas from Computer Vision (e.g., ResNets) applying to Speech; Speech ideas applying to NLP.
- Practitioners should review literature from diverse domains for inspiration.

- **Intuition Staleness:**

- Optimal hyperparameter settings are not static. They can become "stale" due to:
 - Changes in data distributions over time.
 - Server/infrastructure upgrades.
 - Algorithm updates.
- *Recommendation:* Re-evaluate and re-verify hyperparameter settings at least once every few months.

2. Search Strategies

The method chosen for tuning depends heavily on the ratio of model size/data volume to available computational power (CPUs/GPUs).

A. The "Panda" Approach (Babysitting One Model)

This approach involves meticulously managing the training of a single model over an extended period.

- **Context:**

- **Low computational resources:** You do not have enough capacity to train multiple models simultaneously.
- **Massive Data/Models:** Common in online advertising or complex

computer vision tasks where training is resource-intensive.

- **Process:**

1. Initialize parameters (Day 0).
2. Monitor the learning curve (cost function J or validation error) daily.
3. **Intervene manually:** If learning slows or diverges, pause and adjust parameters (e.g., nudge the learning rate α up or down, adjust momentum).
4. Continue "babysitting" the model day-by-day until convergence.

- **Analogy: Pandas** typically have only one child at a time and invest immense effort into ensuring that single offspring survives.

B. The "Caviar" Approach (Training Many Models in Parallel)

This approach involves training many distinct models simultaneously with different hyperparameter settings to find the best performer.

- **Context:**

- **High computational resources:** You have sufficient resources to run many parallel experiments.

- **Process:**

1. Define a set of different hyperparameter configurations.
2. Launch training for all models simultaneously.
3. Allow them to run without manual intervention.
4. Analyze the resulting learning curves (cost function J or error metrics).
5. Select the best-performing model/hyperparameters.

- **Analogy: Fish (Caviar)** lay thousands of eggs (models) at once. They do not tend to individual eggs but rely on the probability that at least one will survive and thrive.

Summary of Differences

Feature	Panda Approach (Babysitting)	Caviar Approach (Parallel)
Model Count	1 (or very few)	Many
Resource Req.	Low	High
Method	Manual daily adjustments	Automated parallel runs
Strategy	Nurture one model to success	Filter for the best survivor
